

NITTE MEENAKSHI INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION, AFFILIATED TO VISVESVARAYA TECHNOLOGICAL UNIVERSITY,
BELGAUM, APPROVED BY AICTE & GOVT.OF KARNATAKA)



Course: Theory of Computation (22ISESC455)

LEARNING ACTIVITY – 2

SIMULATION AND MODELING PROJECT REPORT

on

“Determine whether a given DFA and NFA are equivalent”

Submitted in partial fulfilment of the requirement for the award of Degree of

Bachelor of Engineering

in

Information Science & Engineering

Submitted by:

Aditi.M.Jambha

1NT24IS012

Under the Guidance of

Mrs. Deepthi K

Assistant Professor, Dept. of ISE, NMIT

**Department of Information Science and Engineering
(Accredited by NBA Tier-1)**

2025-2026

(AN AUTONOMOUS INSTITUTION, AFFILIATED TO VISVESVARAYA TECHNOLOGICAL UNIVERSITY, BELGAUM)

Department of Information Science and Engineering
(Accredited by NBA Tier-1)



CERTIFICATE

This is to certify that the Learning Activity – 2, Simulation and Modelling Project Report on **“Determine whether a given DFA and NFA are equivalent”** for the course **Theory of Computation (22ISESC455)** is an authentic work carried out by **Aditi.M.Jambha (1NT24IS012)** Bonafide student of Nitte Meenakshi Institute of Technology, Bangalore in partial fulfilment for the award of the degree of BE in Information Science & Engineering of Visvesvaraya Technological University, Belagavi during the academic year 2025-2026. It is certified that all corrections and suggestions indicated during the internal assessment has been incorporated in the report deposited in the department library. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said course.

Signature of Faculty

Mrs. Deepthi K
Assistant Professor
Department of ISE, NMIT

Signature of HOD

Dr. Mohan S G
Professor & Head
Department of ISE, NMIT

Abstract

This project focuses on determining the equivalence between a Deterministic Finite Automaton (DFA) and a Non-Deterministic Finite Automaton (NFA), which are fundamental computational models in Theory of Computation. Although DFAs and NFAs differ in structure and transition mechanisms, they are known to recognize the same class of languages, called regular languages. The objective of this project is to verify whether a given DFA and NFA accept the same language. The approach involves converting the NFA into an equivalent DFA using subset construction and then comparing it with the given DFA. Equivalence checking is performed by analyzing their minimized forms or by testing acceptance of strings. The implementation is carried out using simulation tools and programming techniques. The results demonstrate that equivalence can be effectively verified using systematic procedures, thereby reinforcing theoretical concepts through practical application.

Acknowledgement

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crowned our effort with success. I express my sincere gratitude to our Principal **Dr. H. C. Nagaraj**, Nitte Meenakshi Institute of Technology for providing facilities.

We wish to thank our HoD, **Dr. Manoj Kumar** for the excellent environment created to further educational growth in our college. We also thank him for the invaluable guidance provided which has helped in the creation of a better project.

I hereby like to thank our *Facutly Mrs. Deepthi K, Assistant Professor*, Department of Information Science & Engineering on their periodic inspection, time to time evaluation of the project and help to bring the project to the present form.

We also thank all our friends, teaching and non-teaching staff at NMIT, Bangalore, for all the direct and indirect help provided in the completion of the project.

Table of Contents

Abstract

Acknowledgement

Sl.no	Chapter Title	Page Number
1	Introduction	6
2	Problem Statement	7
3	Theoretical Background	8-10
4	Methodology	11-12
5	Simulation and Implementation	13-14
6	Testing	15
7	Results, Discussion and Snapshots	16
8	Conclusion	17

List of Figures

References

INTRODUCTION

Theory of Computation is a core area of computer science that studies abstract computational models and their capabilities. One of the most fundamental models is Finite Automata, which are used to recognize patterns and define regular languages. These models have practical applications in areas such as compiler design, text processing, and digital systems.

Finite Automata are broadly classified into Deterministic Finite Automata (DFA) and Non-Deterministic Finite Automata (NFA). A DFA has exactly one transition for each input symbol from a given state, making it simple and predictable. In contrast, an NFA can have multiple transitions for the same input symbol and may include ϵ -transitions, allowing greater flexibility in design.

Even though DFAs and NFAs differ structurally, they are equivalent in terms of computational power. Both recognize the same class of languages, known as regular languages. This equivalence means that any NFA can be converted into an equivalent DFA that accepts the same set of strings.

The main objective of this project is to determine whether a given DFA and NFA are equivalent. This is done by converting the NFA into a DFA using the subset construction method and then comparing it with the given DFA. If both automata accept the same language, they are considered equivalent.

Understanding this concept is important in practical applications like compiler design and system verification, where correctness and consistency of pattern recognition are essential. This project helps in connecting theoretical concepts with implementation, providing a clearer understanding of automata equivalence.

Finite automata are widely used in software and hardware systems for recognizing patterns and validating input sequences. In compiler design, finite automata help in token recognition during lexical analysis. They are also used in communication protocols, control systems, and text editors for efficient pattern searching and validation tasks.

PROBLEM STATEMENT

Finite Automata can be represented using Deterministic Finite Automata (DFA) and Non-Deterministic Finite Automata (NFA). Although both models are theoretically equivalent in recognizing regular languages, their structural differences make direct comparison difficult.

The problem addressed in this project is to determine whether a given DFA and NFA are equivalent, i.e., whether they accept the same set of input strings over a given alphabet. Since an NFA can have multiple transitions for a single input symbol (including ϵ -transitions), it cannot be directly compared with a DFA.

To solve this, the NFA is first converted into an equivalent DFA using the subset construction method. Once both automata are in deterministic form, their equivalence can be verified by comparing their behaviour for different input strings or by analysing their structure.

Objectives:-

- Convert NFA to equivalent DFA
- Compare it with the given DFA
- Verify if both accept the same language

This problem is significant in applications like compiler design and system verification, where ensuring consistent behaviour of automata is essential.

Equivalence checking helps verify whether two automata represent the same logic or language. This is important in optimization problems, compiler verification, and digital circuit design where multiple models may exist for the same system.

THEORETICAL BACKGROUND

Deterministic Finite Automaton (DFA)

A DFA is defined as:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where:

- Q = set of states
- Σ = input alphabet
- δ = transition function
- q_0 = start state
- F = set of final states

Non-Deterministic Finite Automaton (NFA)

An NFA is defined as:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where δ maps to a set of states.

Equivalence of DFA and NFA

- Every NFA has an equivalent DFA
- Both recognize regular languages

DFA explanation:

- DFA processes one symbol at a time.
- At every step only one next state exists.
- No ambiguity during execution.

NFA explanation:

- NFA may move to several states simultaneously.
- ϵ -transitions allow movement without consuming input symbols.
- NFAs are easier to design for complex languages.

Subset Construction Algorithm

Used to convert NFA $\rightarrow$ DFA:

- Each DFA state represents a set of NFA states
- Start state = ϵ -closure of initial state
- Transitions computed for each input symbol

Steps in Subset Construction

Step 1: Find the Start State

The start state of the DFA is obtained from the ϵ -closure of the initial state of the NFA.

- ϵ -closure of a state means all states reachable using ϵ -transitions without consuming any input symbol.
- If there are no ϵ -transitions, the start state is simply the initial NFA state.

Example:

If the NFA start state is q_0 , then:

DFA Start State = ϵ -closure(q_0)

Step 2: Compute Transitions

For every DFA state and every input symbol:

1. Find all possible NFA states reachable using that input symbol.
2. Combine those states into a single DFA state.
3. Repeat the process for newly formed states.

Example:

If from states $\{q_0, q_1\}$ on input 0, the NFA reaches q_2 and q_3 , then:

$$\delta(\{q_0, q_1\}, 0) = \{q_2, q_3\}$$

This subset becomes a new DFA state.

Step 3: Create New DFA States

Whenever a new subset of NFA states is found, it is treated as a new DFA state.

For example:

DFA State Represents NFA States

A $\{q_0\}$

B $\{q_0, q_1\}$

C $\{q_2, q_3\}$

Step 4: Identify Final States

A DFA state is considered an accepting state if at least one of the NFA states in the subset is a final state.

Example:

If q_2 is a final state in the NFA, then any DFA state containing q_2 will also be a final state.

Step 5: Continue Until No New States Are Generated

The process continues until:

- all subsets are processed, and
- no additional DFA states can be formed.

The final DFA obtained will behave exactly like the original NFA.

METHODOLOGY

The methodology followed in this project involves analyzing the given DFA and NFA, converting the NFA into an equivalent DFA, and then comparing both automata to determine whether they recognize the same language.

Initially, the states, input symbols, start state, final states, and transition functions of the given DFA and NFA were identified. The DFA was studied directly since it contains exactly one transition for each input symbol from every state.

Next, the NFA was converted into an equivalent DFA using the subset construction method. In this method, each state of the new DFA represents a set of states from the NFA. The process began with the start state of the NFA and continued by finding all possible state combinations for each input symbol until no new states were generated.

After obtaining the equivalent DFA, transition tables and state diagrams were prepared for both automata. Sample input strings were then tested on both machines. The sequence of state transitions was traced carefully, and the final state reached after processing each string was observed.

If both automata accepted and rejected the same set of input strings, they were considered equivalent. The entire methodology helped in understanding automata conversion, state transitions, and equivalence checking in finite automata.

Steps Followed

1. Define the given DFA and NFA
2. Construct transition tables
3. Convert NFA into DFA using subset construction
4. Draw the equivalent DFA
5. Test different input strings
6. Compare outputs of DFA and NFA
7. Verify equivalence of both automata

Algorithm

Step 1:

Read the states, input symbols, start state, and final states of DFA and NFA.

Step 2:

Construct transition tables for both automata.

Step 3:

Apply subset construction:

- Start with the initial NFA state

- Find reachable states for each input
- Create equivalent DFA states

Step 4:

Draw the converted DFA.

Step 5:

Provide sample input strings

Step 6:

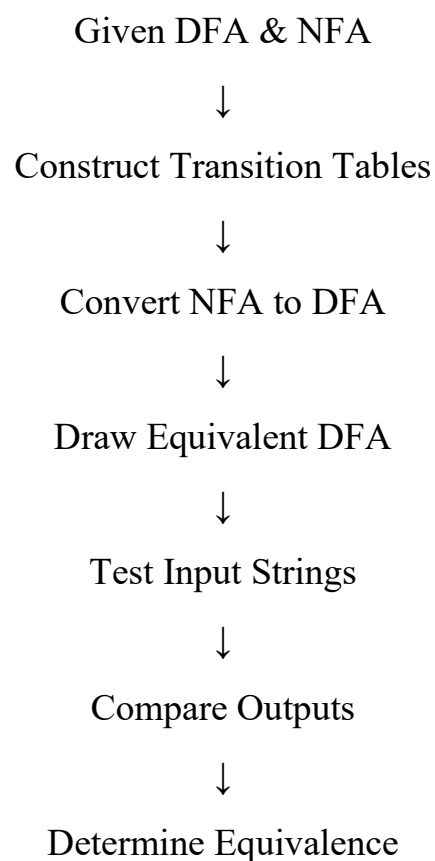
Trace state transitions for each string.

Step 7:

Compare results of DFA and converted DFA.

Step 8:

If both automata produce identical outputs for all test cases, conclude that they are equivalent.



SIMULATION AND IMPLEMENTATION

In this project, the given DFA and NFA were analyzed to determine whether they are equivalent. The simulation was carried out by representing both automata using state diagrams and transition tables.

First, the DFA was examined by tracing the movement from one state to another for different input symbols. Since a DFA has only one transition for each input symbol, the next state for every input was easily determined.

Next, the NFA was analyzed. Since an NFA may have multiple transitions for the same input symbol, it was converted into an equivalent DFA using the subset construction method. In this method, each state of the new DFA represents a set of states of the NFA.

After conversion, both DFAs were tested with different input strings. For each string, the sequence of state transitions was traced and the final state was checked. If the final state was an accepting state, the string was accepted; otherwise, it was rejected.

The results obtained from both automata were compared. Since both automata showed the same acceptance and rejection behavior for the tested strings, they were considered equivalent.

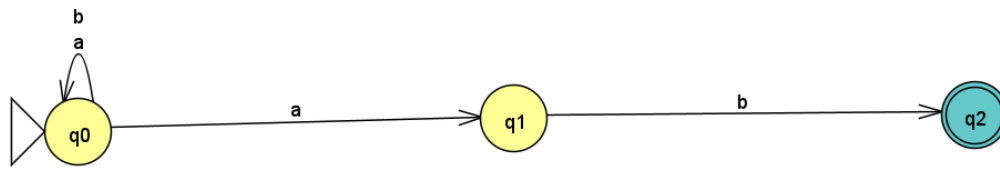
This simulation helped in understanding the practical working of DFA, NFA, state transitions, and equivalence checking.

During simulation, every input string was processed sequentially. For each symbol, the transition function determined the next state. In the converted DFA, subset states represented combinations of original NFA states, ensuring deterministic execution.

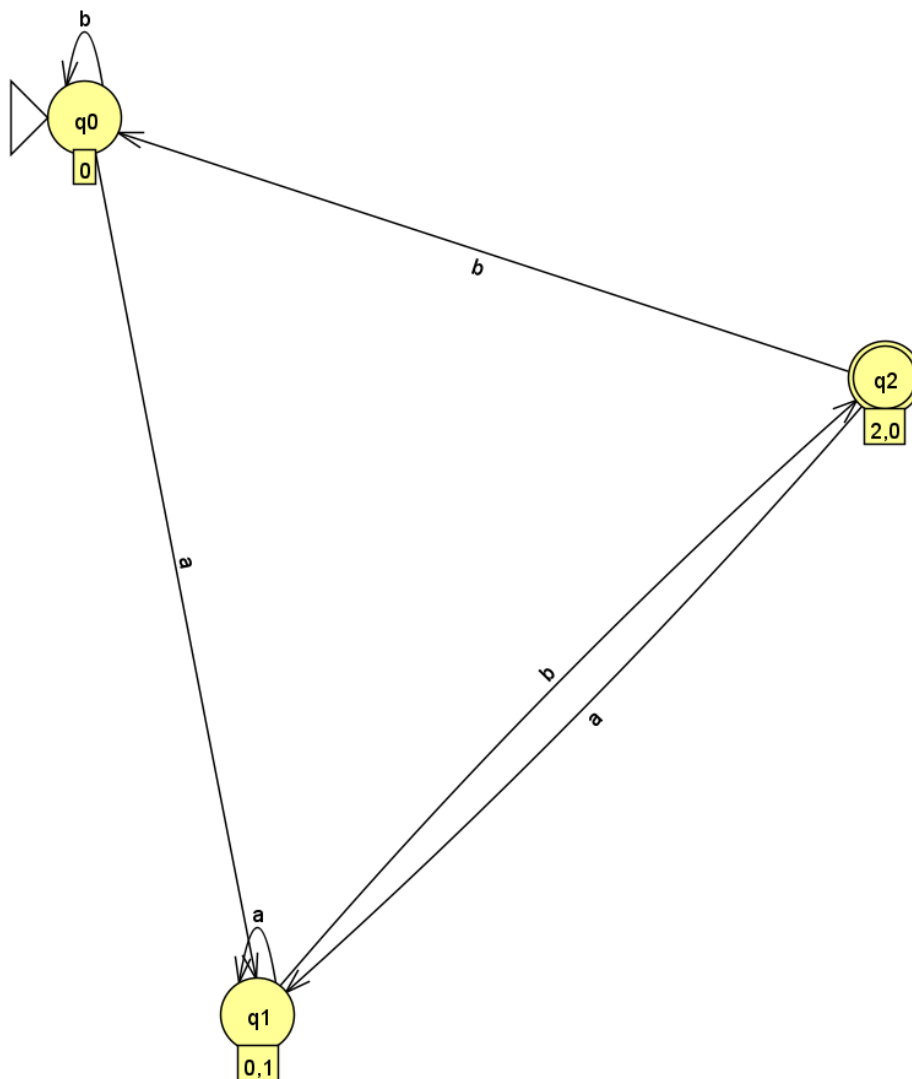
The implementation of DFA and NFA equivalence checking was carried out using Python programming language. The NFA was converted into an equivalent DFA using subset construction logic, and both automata were tested with multiple input strings. The outputs obtained from both automata were compared to verify language equivalence. Console outputs and transition traces were used to analyse acceptance and rejection behaviour.

Implementation on JFLAP:-

(i)NFA



(ii) DFA



TESTING

Testing is an important step used to verify whether the given DFA and NFA are equivalent. After converting the NFA into an equivalent DFA, both automata were tested with different input strings. The purpose of testing was to check whether both automata accept or reject the same strings.

A set of sample strings over the given input alphabet was selected. Each string was given as input to the DFA and the converted DFA. The state transitions were traced step by step, and the final state reached after processing the complete string was observed. If the final state belonged to the set of accepting states, the string was accepted; otherwise, it was rejected.

The outputs of both automata were compared for every test case. If both automata produced identical results for all tested strings, they were considered equivalent.

Sl. No	Input String	DFA Output	NFA Output	Result
1	ϵ (empty string)	Reject	Reject	Match
2	0	Reject	Reject	Match
3	1	Accept	Accept	Match
4	01	Accept	Accept	Match
5	11	Accept	Accept	Match
6	101	Accept	Accept	Match
7	001	Accept	Accept	Match

RESULTS, DISCUSSION AND SNAPSHOTS

Results

The NFA was successfully converted into an equivalent DFA using the subset construction method. The converted DFA was then compared with the given DFA by testing multiple input strings.

Both automata accepted and rejected the same strings for all test cases. Hence, it was concluded that the given DFA and NFA recognize the same language.

Discussion

The project clearly shows that DFA and NFA, though different in structure, have the same computational power. A DFA has only one transition for each input symbol, making it easier to implement. An NFA allows multiple transitions, which makes its design simpler for certain languages.

The conversion of NFA into DFA may increase the number of states, but it ensures deterministic behavior. This proves the theoretical concept that every NFA has an equivalent DFA.

The simulation also helped in understanding:

- State transitions
- Accepting and rejecting states
- Subset construction method
- Equivalence checking of automata

CONCLUSION

This project successfully demonstrated the process of determining whether a given Deterministic Finite Automaton (DFA) and Non-Deterministic Finite Automaton (NFA) are equivalent. The study focused on understanding the structural differences between the two automata and verifying that both recognize the same language.

The given NFA was converted into an equivalent DFA using the subset construction method. During this process, the possible combinations of NFA states were identified and represented as states in the new DFA. After conversion, transition tables and state diagrams were prepared to clearly represent the behavior of both automata.

Several input strings were tested on the given DFA and the converted DFA. The sequence of state transitions and the final states reached for each string were carefully analyzed. The results showed that both automata accepted and rejected the same set of strings. Hence, it was concluded that the given DFA and NFA are equivalent.

The project also provided practical understanding of important concepts in Theory of Computation such as finite automata, state transitions, accepting states, subset construction, and equivalence checking. It helped in connecting theoretical knowledge with practical simulation and analysis.

One important observation made during the project was that although NFAs are easier to design, their equivalent DFAs may contain a larger number of states. However, DFAs are simpler to execute because they have only one transition for each input symbol. This highlights the trade-off between simplicity of design and ease of implementation.

Overall, the project improved the understanding of regular languages and automata theory. Future work can include DFA minimization, implementation using software tools, and equivalence checking for more complex automata and grammars.

REFERENCES

1. John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*.
2. Peter Linz, *An Introduction to Formal Languages and Automata*.
3. Theory of Computation Lecture Notes, VTU.
4. Class Notes and Laboratory Manual.